

OS Skill 16

16(a) Background Jobs

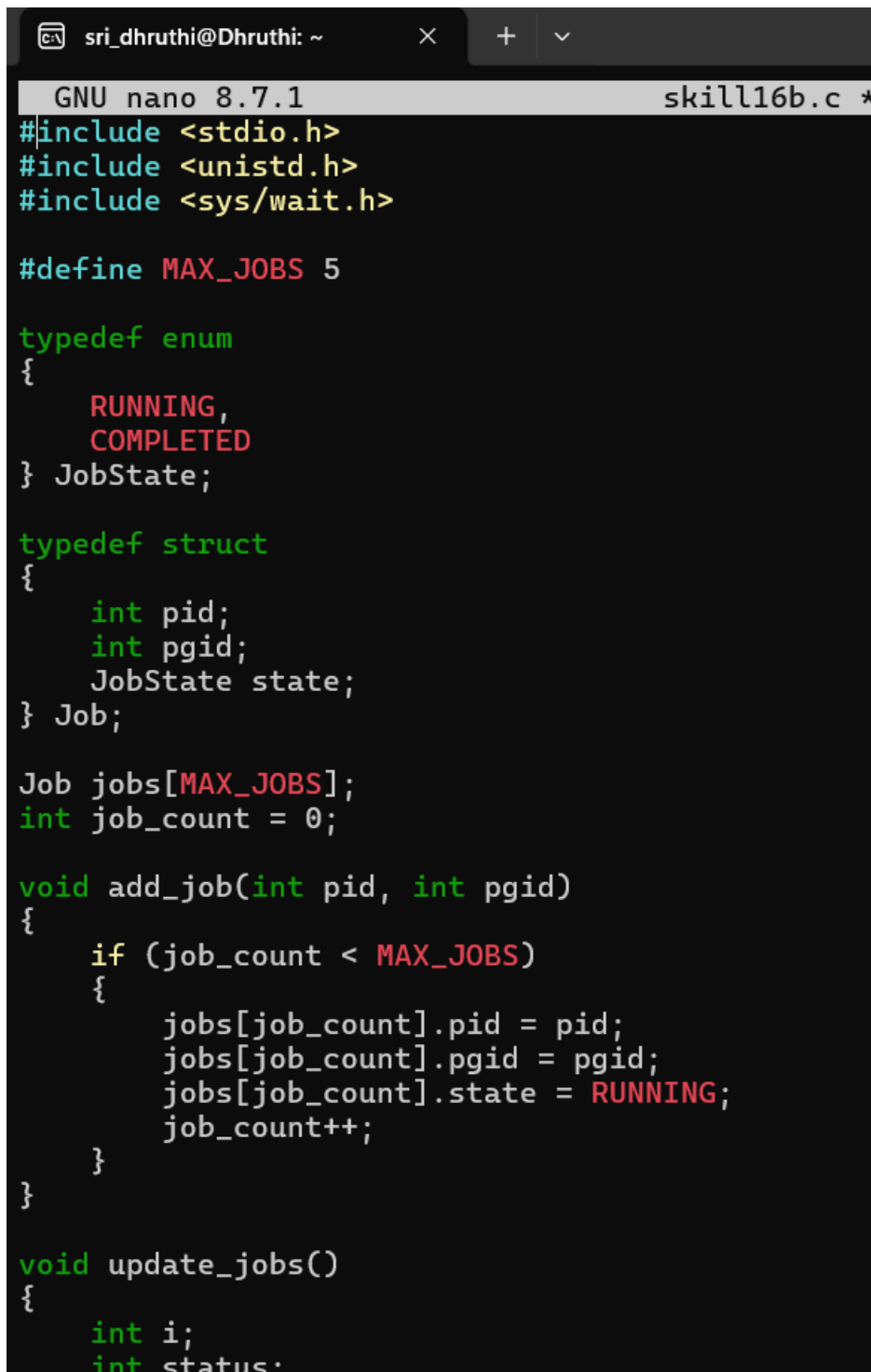
What it does: Starts a process in the background without blocking the terminal and monitors when it finishes.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 skill16a.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        printf("Background job started. PID = %d\n", getpid());  
  
        sleep(5);  
  
        printf("Background job completed. PID = %d\n", getpid());  
        exit(0);  
    }  
  
    printf("Parent: Background job launched. PID = %d\n", pid);  
    printf("Parent: Prompt is available immediately.\n");  
  
    /* Non-blocking check */  
    waitpid(pid, &status, WNOHANG);  
  
    printf("Parent continues working...\n");  
  
    sleep(6);  
}
```

```
Netto Linux Pipeline  
sri_dhruthi@Dhruthi:~$ nano skill16a.c  
sri_dhruthi@Dhruthi:~$ gcc skill16a.c -o skill16a  
sri_dhruthi@Dhruthi:~$ ./skill16a  
Parent: Background job launched. PID = 1069  
Parent: Prompt is available immediately.  
Parent continues working...  
Background job started. PID = 1069  
Background job completed. PID = 1069  
Background job finished successfully.
```

16(b) Job Table

What it does: Maintains a job table containing process IDs and states, updates running jobs, and removes completed jobs.



```
GNU nano 8.7.1 skill16b.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 5

typedef enum
{
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int pid;
    int pgid;
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(int pid, int pgid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].pgid = pgid;
        jobs[job_count].state = RUNNING;
        job_count++;
    }
}

void update_jobs()
{
    int i;
    int status;
```

```

int i;
int status;

for (i = 0; i < job_count; i++)
{
    if (jobs[i].state == RUNNING)
    {
        if (waitpid(jobs[i].pid, &status, WNOHANG) > 0)
        {
            jobs[i].state = COMPLETED;
        }
    }
}

void remove_completed_jobs()
{
    int i, j;

    for (i = 0; i < job_count;)
    {
        if (jobs[i].state == COMPLETED)
        {
            for (j = i; j < job_count - 1; j++)
            {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

```

```

int main()
{
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0)
    {
        sleep(3);
        return 0;
    }

    add_job(pid1, pid1);

    pid2 = fork();

    if (pid2 == 0)
    {
        sleep(5);
        return 0;
    }

    add_job(pid2, pid2);

    show_jobs();

    sleep(4);

    update_jobs();

    printf("\nAfter status update:\n");
}

```

```

    add_job(pid2, pid2);

    show_jobs();

    sleep(4);

    update_jobs();

    printf("\nAfter status update:\n");
    show_jobs();

    remove_completed_jobs();

    printf("\nAfter removing completed jobs:\n");
    show_jobs();

    while (job_count > 0)
    {
        update_jobs();
        remove_completed_jobs();
        sleep(1);
    }

    printf("\nAll jobs completed and removed.\n");

    return 0;
}

```

Background job finished successfully.

```

sri_dhruthi@Dhruthi:~$ nano skill16b.c
sri_dhruthi@Dhruthi:~$ gcc skill16b.c -o skill16b
sri_dhruthi@Dhruthi:~$ ./skill16b

```

Job Table:

```

PID: 1134 | PGID: 1134 | State: RUNNING
PID: 1135 | PGID: 1135 | State: RUNNING

```

After status update:

Job Table:

```

PID: 1134 | PGID: 1134 | State: COMPLETED
PID: 1135 | PGID: 1135 | State: RUNNING

```

After removing completed jobs:

Job Table:

```

PID: 1135 | PGID: 1135 | State: RUNNING

```

All jobs completed and removed.

```

sri_dhruthi@Dhruthi:~$ |

```